***Q9: Network flow issues come up in dealing with natural disasters and other crises, since major unexpected events often require the movement and evacuation of large numbers of people in a short amount of time.

Consider the following scenario. Due to large-scale flooding in a re-gion, paramedics have identified a set of n injured people distributed across the region who need to be rushed to hospitals. There are k hos-pitals in the region, and each of the n people needs to be brought to a hospital that is within a half-hour's driving time of their current location (so different people will have different options for hospitals, depending on where they are right now).

At the same time, one doesn't want to overload any one of the hospitals by sending too many patients its way. The paramedics are in touch by cell phone, and they want to collectively work out whether they can choose a hospital for each of the injured people in such a way that the load on the hospitals is balanced: Each hospital receives at most ⌈n/k⌉ people.

Give a polynomial-time algorithm that takes the given information about the people's locations and determines whether this is possible.

Problem Recap:

There are n injured people, each needing to be assigned to one hospital within a 30-minute drive.

There are k hospitals, and no hospital can be assigned more than ⌈n/k⌉ people.

The task: efficiently decide whether it is possible to assign each person to a hospital within their reach such that no hospital is overloaded.

Core Idea:

We can model this as a flow network, so we can think of the people and hospitals as two columns of nodes.

1.We can create a "source" node that sends "people units" to the people nodes. We connect each person to the hospitals they can reach.

2.We can create a "sink" node that receives "people units" from the hospitals.

3.The hospital nodes have a capacity, that representing the maximum number of people they can take. If the maximum flow from the source to the sink is exactly n, then we have successfully assigned all n people. If it's less than n, then it is not possible.

Diagram:

[Source Node s]

| (Capacity: 1 per edge)

v

[People Nodes] -> [Hospital Nodes] -> [Sink Node t]

P1 --------------------> H1 (Capacity: ceil(n/k)) --->

P2 --------------------> H2 (Capacity: ceil(n/k)) --->

Pn --------------------> Hk (Capacity: ceil(n/k)) --->

Explanation of the Diagram:

Source (s): A single starting point.

People Nodes (P1, P2, ..., Pn): Each represents one injured person.

Edges from s to each Pi have a capacity of 1. This ensures each person is only assigned once.

Hospital Nodes (H1, H2, ..., Hk): Each represents a hospital.

Edges from each Hj to the sink t have a capacity of ⌈n/k⌉. This enforces the load-balancing constraint.

Middle Edges (Pi -> Hj): An edge exists from person Pi to hospital Hj if and only if Hj is within a half-hour drive of Pi's location.

These edges have a capacity of 1. (A person can be sent to a hospital at most once, but this is already enforced by the source edges).

Algorithm Construction:

Input:

A list of n people. For each person i, you have a list Hi of hospitals they can reach.

The number of hospitals k.

Output: "YES" if a balanced assignment is possible, "NO" otherwise.

Steps:

Create a Flow Graph:

Create a source node s and a sink node t.

Create a node for each of the n people.

Create a node for each of the k hospitals.

Calculate the capacity for each hospital: C = ⌈n/k⌉.

Add Edges:

For each person i (from 1 to n):

Add an edge from s to person_i with capacity 1.

For each hospital j (from 1 to k):

Add an edge from hospital_j to t with capacity C.

For each person i and for each hospital j in person i's list of reachable hospitals H_i:

Add an edge from person_i to hospital_j with capacity 1.

Compute Maximum Flow:

We can run on a standard polynomial-time Max-Flow algorithm (like the Edmonds-Karp algorithm, which is based on Ford-Fulkerson with BFS) on the graph , with source s and sink t.

Result:

Let F be the value of the maximum flow computed in the previous step.

If F == n:

Output "YES". A valid, balanced assignment exists.

Else (if F < n):

Output "NO". It is impossible to assign all people without violating the constraints.

Time Complexity Analysis:

The graph has O(n + k) nodes, which is O(n) since k ≤ n in a practical scenario.

The number of edges is: n edges from the source. k edges to the sink.

At most n * k edges between people and hospitals. In the worst case, this is O(n^2).

Using the Edmonds-Karp algorithm, which runs in $O(V E^2)$ time, the overall worst-case complexity is $O((n+k) * (n k)^2)$, which simplifies to $O(n^5)$ in the worst case. While this is polynomial, it's high.

```
from collections import deque
import math
```

```
class FlowGraph:
    def __init__(self, n):
        self.n = n  # number of nodes
        self.graph = [[] for _ in range(n)]
        self.capacity = [[0] * n for _ in range(n)]

    def add_edge(self, u, v, cap):
        self.graph[u].append(v)# Add forward edge
        self.capacity[u][v] += cap

        self.graph[v].append(u)# Add backward edge for residual graph
        self.capacity[v][u] = 0

    def bfs(self, s, t, parent):
        visited = [False] * self.n
        queue = deque()
        queue.append(s)
        visited[s] = True

        while queue:
            u = queue.popleft()

            for v in self.graph[u]:
                if not visited[v] and self.capacity[u][v] > 0:
                    visited[v] = True
                    parent[v] = u
                    if v == t:
                        return True
                    queue.append(v)
        return False
```

```
            return False

        def edmonds_karp(self, source, sink):
            parent = [-1] * self.n
            max_flow = 0

            while self.bfs(source, sink, parent):
                # Find bottleneck capacity
                path_flow = float('inf')
                v = sink
                while v != source:
                    u = parent[v]
                    path_flow = min(path_flow, self.capacity[u][v])
                    v = u

                # Update residual capacities
                v = sink
                while v != source:
                    u = parent[v]
                    self.capacity[u][v] -= path_flow
                    self.capacity[v][u] += path_flow
                    v = u

                max_flow += path_flow

            return max_flow
```

```
    def add_edge(self, u, v, cap):
            self.graph[u].append(v)# Add forward edge
            self.capacity[u][v] += cap

            self.graph[v].append(u)# Add backward edge for residual graph
            self.capacity[v][u] = 0
```

```
    def bfs(self, s, t, parent):
            visited = [False] * self.n
            queue = deque()
            queue.append(s)
            visited[s] = True

            while queue:
                u = queue.popleft()

                for v in self.graph[u]:
                    if not visited[v] and self.capacity[u][v] > 0:
                        visited[v] = True
                        parent[v] = u
                        if v == t:
                            return True
                        queue.append(v)
            return False
```

```
     def edmonds_karp(self, source, sink):
            parent = [-1] * self.n
            max_flow = 0

            while self.bfs(source, sink, parent):
                # Find bottleneck capacity
                path_flow = float('inf')
                v = sink
                while v != source:
                    u = parent[v]
                    path_flow = min(path_flow, self.capacity[u][v])
                    v = u

                # Update residual capacities
                v = sink
                while v != source:
                    u = parent[v]
                    self.capacity[u][v] -= path_flow
                    self.capacity[v][u] += path_flow
                    v = u

                max_flow += path_flow
```

```
            return max_flow
```

```
Check if all n people can be assigned to k hospitals with load balancing.

Args:
    n: number of people
    k: number of hospitals
    person_hospitals: list of lists, where person_hospitals[i] contains
                      indices of hospitals that person i can reach (0-indexed)

Returns:
    True if assignment is possible, False otherwise
```

```
    def can_assign_people(n, k, person_hospitals):

        # Calculate hospital capacity
        hospital_capacity = math.ceil(n / k)

        # Node indices:
        # 0: source
        # 1 to n: people nodes
        # n+1 to n+k: hospital nodes
        # n+k+1: sink
        source = 0
        sink = n + k + 1
        total_nodes = n + k + 2

        graph = FlowGraph(total_nodes)

        # Connect source to people (capacity 1)
        for person in range(1, n + 1):
            graph.add_edge(source, person, 1)

        # Connect people to reachable hospitals (capacity 1)
        for person in range(n):
            person_node = person + 1
            for hospital in person_hospitals[person]:
                hospital_node = n + 1 + hospital
                graph.add_edge(person_node, hospital_node, 1)

        # Connect hospitals to sink (capacity = hospital_capacity)
        for hospital in range(k):
            hospital_node = n + 1 + hospital
            graph.add_edge(hospital_node, sink, hospital_capacity)

        # Compute max flow
        max_flow = graph.edmonds_karp(source, sink)

        return max_flow == n
```

```
If assignment is possible, return the actual assignment.

Returns:
    List where assignment[i] is the hospital assigned to person i,
    or None if no assignment is possible.
```

```
    def get_assignment(n, k, person_hospitals):

        hospital_capacity = math.ceil(n / k)

        source = 0
        sink = n + k + 1
        total_nodes = n + k + 2

        graph = FlowGraph(total_nodes)

        # Build the same graph as before
```

```
    for person in range(1, n + 1):
        graph.add_edge(source, person, 1)

    for person in range(n):
        person_node = person + 1
        for hospital in person_hospitals[person]:
            hospital_node = n + 1 + hospital
            graph.add_edge(person_node, hospital_node, 1)

    for hospital in range(k):
        hospital_node = n + 1 + hospital
        graph.add_edge(hospital_node, sink, hospital_capacity)

    # Compute max flow
    max_flow = graph.edmonds_karp(source, sink)

    if max_flow != n:
        return None

    # Reconstruct assignment from residual graph
    assignment = [-1] * n
    for person in range(n):
        person_node = person + 1
        for hospital in range(k):
            hospital_node = n + 1 + hospital
            # If there's flow from hospital to person in residual graph,
            # it means person is assigned to that hospital
            if graph.capacity[hospital_node][person_node] > 0:
                assignment[person] = hospital
                break

    return assignment
```

Example

```
if __name__ == "__main__":
    print("=== Hospital Assignment Problem ===\n")

    # Test Case 1: Simple case that should work
    print("Test Case 1: Check for possible case")
    n1, k1 = 4, 2
    # Person 0 can go to hospitals 0,1; Person 1 to hospital 0; Person 2 to hospital 1; Person 3 to hospital 1
    person_hospitals1 = [[0, 1], [0], [1], [1]]

    possible1 = can_assign_people(n1, k1, person_hospitals1)
    assignment1 = get_assignment(n1, k1, person_hospitals1)

    print(f"People: {n1}, Hospitals: {k1}, Capacity per hospital: {math.ceil(n1/k1)}")
    print(f"Person-hospital reachability: {person_hospitals1}")
    print(f"Assignment possible: {possible1}")
    print(f"Actual assignment: {assignment1}")
    print()

    # Test Case 2: Should be impossible (hospital 0 overloaded)
    print("Test Case 2: Check for impossible case")
    n2, k2 = 4, 2
    # All people can only go to hospital 0
    person_hospitals2 = [[0], [0], [0], [0]]

    possible2 = can_assign_people(n2, k2, person_hospitals2)
    assignment2 = get_assignment(n2, k2, person_hospitals2)

    print(f"People: {n2}, Hospitals: {k2}, Capacity per hospital: {math.ceil(n2/k2)}")
    print(f"Person-hospital reachability: {person_hospitals2}")
    print(f"Assignment possible: {possible2}")
    print(f"Actual assignment: {assignment2}")
    print()

    # Test Case 3: More complex case
    print("Test Case 3: Check for Complex case")
    n3, k3 = 6, 3
    person_hospitals3 = [
        [0, 1],    # Person 0 can go to hospitals 0 or 1
        [0, 2],    # Person 1 can go to hospitals 0 or 2
```

```
        [1, 2],    # Person 2 can go to hospitals 1 or 2
        [0],       # Person 3 can only go to hospital 0
        [1],       # Person 4 can only go to hospital 1
        [2]        # Person 5 can only go to hospital 2
    ]

    possible3 = can_assign_people(n3, k3, person_hospitals3)
    assignment3 = get_assignment(n3, k3, person_hospitals3)

    print(f"People: {n3}, Hospitals: {k3}, Capacity per hospital: {math.ceil(n3/k3)}")
    print(f"Person-hospital reachability: {person_hospitals3}")
    print(f"Assignment possible: {possible3}")
    print(f"Actual assignment: {assignment3}")

    # Verify the assignment
    if assignment3:
        hospital_counts = [0] * k3
        for person, hospital in enumerate(assignment3):
            hospital_counts[hospital] += 1
            # Check if assignment is valid
            if hospital not in person_hospitals3[person]:
                print(f"ERROR: Person {person} assigned to unreachable hospital {hospital}")

        print(f"Hospital assignment counts: {hospital_counts}")
        print(f"All within capacity: {all(count <= math.ceil(n3/k3) for count in hospital_counts)}")
```

```
=== Hospital Assignment Problem ===

Test Case 1: Check for possible case
People: 4, Hospitals: 2, Capacity per hospital: 2
Person-hospital reachability: [[0, 1], [0], [1], [1]]
Assignment possible: True
Actual assignment: [0, 0, 1, 1]

Test Case 2: Check for impossible case
People: 4, Hospitals: 2, Capacity per hospital: 2
Person-hospital reachability: [[0], [0], [0], [0]]
Assignment possible: False
Actual assignment: None

Test Case 3: Check for Complex case
People: 6, Hospitals: 3, Capacity per hospital: 2
Person-hospital reachability: [[0, 1], [0, 2], [1, 2], [0], [1], [2]]
Assignment possible: True
Actual assignment: [0, 2, 1, 0, 1, 2]
Hospital assignment counts: [2, 2, 2]
All within capacity: True
```